

BickSpec (.bks)

Especificación del lenguaje y estado del proyecto - Fase III

Fecha: 07 de Mayo de 2026

Este documento presenta una versión ampliada y detallada de la documentación de BickSpec. Mantiene el formato general de los documentos de Fase I y Fase II, pero amplía considerablemente el contenido para describir con mayor profundidad la evolución completa del proyecto, la arquitectura del compilador, las categorías de errores, el flujo de traducción, el runtime financiero y la estructura final de salida del compilador.

Componente	Estado en Fase III
Gramática ANTLR4 (.g4)	Implementada, validada y reutilizada desde fases previas.
Analizador léxico	Funcional, con diagnóstico estandarizado de errores.
Parser / análisis sintáctico	Funcional y estable para casos válidos e inválidos.
Tabla de símbolos	Implementada con exportación CSV.
Validación semántica	Activa y con bloqueo de generación ante error.
Generación de Java	Automática y más completa que en Fase II.
Build del Java generado	Integrado al flujo final.
Execution del programa generado	Integrada para programas válidos.
Jar ejecutable final	bickspec-compiler-1.0.0.jar

1. Equipo y repositorio

El proyecto fue desarrollado colaborativamente manteniendo versionamiento en GitHub y una evolución incremental del compilador por fases.

Nombre	Carné
Paolo Marroquin	1146822
Zane Lockridge	1373724
Martin Sierra	1244523

Repositorio del proyecto: <https://github.com/paolomarroq/bickspec-lang>

2. Resumen detallado de Fase I

La Fase I se concentró en la definición del lenguaje y del analizador léxico. El entregable clave fue la gramática base en formato ANTLR4 y la documentación de las reglas léxicas, símbolos terminales y no terminales.

- Definición del archivo base de gramática .g4.
- Diseño de tokens como palabras reservadas, identificadores, números, strings, operadores y comentarios.
- Establecimiento del formato general del lenguaje con PROJECT, sentencias, funciones y expresiones.
- Implementación del flujo de tokenización para leer archivos .bks y mostrar tokens y lexemas.
- Construcción de la primera suite de pruebas funcionales del lenguaje.

En términos conceptuales, Fase I dejó lista la capa de reconocimiento formal: el lenguaje ya existía como especificación y el proyecto ya tenía una base técnica sobre la cual construir parser, semántica y generación de código en etapas posteriores.

Ejemplo de salida típica de Fase I:

```
PROJECT 'PROJECT'  
STRING_LITERAL '"P1 - Hola Mundo"'  
LBRACE '{'  
DISPLAY 'DISPLAY'  
STRING_LITERAL '"Hola Mundo"'  
RBRACE '}'
```

3. Resumen detallado de Fase II

La Fase II extendió el trabajo anterior hacia el análisis sintáctico y el inicio del análisis semántico. El proyecto dejó de ser solamente un lexer y pasó a validar la estructura completa de programas BickSpec mediante ANTLR.

- Parser ANTLR4 funcional sobre programas válidos e inválidos.
- Detección diferenciada de errores léxicos y sintácticos.
- Generación del parse tree textual.
- Visitor semántico inicial mostrando qué nodo fue visitado y qué acción se planeaba realizar.
- Exportación de árboles de parseo en formatos DOT y SVG.
- Traducción dirigida por la sintaxis inicial hacia Java como evidencia del lenguaje destino.
- Extensión del lenguaje con asignaciones batch para series monetarias y temporales.

En esta fase se consolidó una característica muy importante: el proyecto ya era capaz de recorrer el árbol y asociar reglas del lenguaje con acciones concretas.

Ejemplo de salida típica de Fase II:

```
==== testing/P1_HolaMundo.bks ====
PARSE OK
Parse tree:
(program (projectBlock ...))
Semantic visit trace:
[VISIT] projectBlock -> entering project ...
[ACTION] create Java entry point ...
```

4. Alcance detallado de Fase III

La Fase III cerró el ciclo principal del proyecto. A partir del lenguaje construido en las fases anteriores, el compilador se convirtió en un traductor fuente-a-fuente más completo: lee un archivo .bks, lo analiza léxica, sintáctica y semánticamente, genera un archivo .java, ejecuta el build del Java generado y, para pruebas válidas, puede ejecutar el resultado final.

- Implementación de tabla de símbolos real.
- Bloqueo de generación cuando existen errores léxicos, sintácticos o semánticos.
- Códigos de error estandarizados por categoría.
- Generación automática de Java más completa y coherente.
- Integración de built-ins del lenguaje con helpers de runtime Java.
- Build del .java generado hacia .class.
- Execution del resultado para pruebas válidas.
- Uso del jar final bickspec-compiler como punto de entrada profesional del compilador.

Con esto, el proyecto pasó a mostrar un flujo integral: la entrada del usuario en BickSpec se transforma en una salida ejecutable en Java, manteniendo trazabilidad mediante tablas de símbolos, árboles de parseo y mensajes de estado del compilador.

5. Construcciones y capacidades del lenguaje

El lenguaje BickSpec quedó diseñado para cubrir ejemplos académicos y casos típicos de evaluación económica. Entre las construcciones implementadas y utilizadas se encuentran:

- Declaraciones y asignaciones simples mediante `:=`.
- Asignaciones batch para series monetarias y temporales.
- Literales numéricos, textuales, monetarios y temporales.
- Operadores aritméticos y lógicos.
- Condicionales IF / THEN / ELSE / END.
- Bucles como WHILE y otras estructuras soportadas por la gramática.
- Definición de funciones y llamadas a funciones.
- Importación de módulos o archivos del propio lenguaje.
- Conversión de valores con `to` e `in`.
- Built-ins financieros como `NPV()` y `PAYBACK()`.

6. Flujo técnico completo del compilador

```
Archivo .bks
↓
BickSpecLexer
↓
BickSpecParser
↓
Parse Tree
↓
Validación semántica + Tabla de símbolos
↓
BickSpecJavaTranslatorVisitor
↓
Archivo .java generado
↓
BUILD del Java generado
↓
EXECUTION del programa resultante
```

7. Clases y archivos importantes de Fase III

La siguiente tabla resume los componentes centrales del compilador en su estado actual.

Componente	Responsabilidad principal
TranspileRunner	Entry point final del compilador. Ejecuta parse, semántica, CSV, árbol, Java, build, execution y reporte resumido.
ParseRunner	Runner de validación. Ejecuta parseo, semántica, símbolos y árbol sin generación completa ni execution.
BickSpecParseSupport	Utilidad compartida para resolver archivos de entrada, normalizar rutas, ejecutar lexer/parser y construir ParseResult.
BickSpecSemanticVisitor	Visitor semántico principal. Construye tabla de símbolos y detecta errores semánticos.
BickSpecJavaTranslator	Visitor de generación Java. Traduce el parse tree a un archivo fuente Java legible y coherente.
SymbolTable / SymbolInfo	Modelo de símbolos con nombre, tipo, scope, línea de declaración, inicialización y notas.
SymbolTableCsvExporter	Exporta la tabla de símbolos a CSV en output/symbols/.
CompilerDiagnostic	Modelo uniforme de diagnóstico para errores y mensajes del compilador.
SemanticResult	Contenedor del resultado semántico: éxito, tabla de símbolos, traza y diagnósticos.
ParseTreeGraphGenerator	Exporta el árbol a DOT y SVG en output/trees/.
docs/BickSpec.g4	Gramática ANTLR4 del lenguaje. Base de lexer y parser.
app/pom.xml	Configuración Maven y artefacto final bickspec-compiler.

8. Tabla de símbolos y validación semántica

La Fase III incorpora una semántica práctica orientada a controlar el paso a la generación de código. Solo los programas semántica y estructuralmente válidos generan Java.

Campo	Descripción
name	Nombre del identificador.
type	Tipo semántico inferido o declarado: number, money, module, function, etc.
scope	Ámbito lógico; por ejemplo global o function:.
declared_at	Línea de declaración del símbolo.
initialized	Indica si el símbolo se considera inicializado al momento del análisis.
notes	Metadatos o aclaraciones de uso, por ejemplo import, exchange rate o built-in result.

Ejemplo de CSV exportado:

```
name,type,scope,declared_at,initialized,notes
R,number,global,line 4,true,input variable
CAPEX,money,global,line 10,true,stored internally in USD
NPV_VAL,money,global,line 18,true,built-in result
```

Validaciones semánticas clave:

- Uso de variables antes de su declaración.
- Uso de variables antes de inicialización, cuando aplica.
- Declaraciones duplicadas en el mismo scope.
- Funciones duplicadas o llamadas a funciones no declaradas.
- Errores simples de tipo en asignaciones, condiciones y argumentos.

9. Catálogo detallado de errores y diagnósticos

El proyecto adopta un formato común para los diagnósticos: [ERROR] CÓDIGO - Mensaje at line X:Y.

Código	Categoría	Etapas	Significado
LEX01	Léxico	Lexer	Carácter o token inválido no reconocido por el lexer.
SYN01	Sintaxis	Parser	Estructura inválida según la gramática ANTLR.
SEM01	Semántica	Visitor	Variable usada antes de declaración o antes de inicialización.
SEM02	Semántica	Visitor	Declaración duplicada en el mismo scope: variable, función, import, FX o parámetro.
SEM03	Semántica	Visitor	Error simple de tipo en asignaciones, condiciones o argumentos.
SEM04	Semántica	Visitor	Llamada a función no declarada, excepto built-ins oficiales.
GEN01	Generación	Input	Fallo leyendo el archivo fuente .bks.
GEN02	Generación	CSV	Fallo exportando la tabla de símbolos.
GEN03	Generación	Java	Fallo escribiendo el archivo .java.
GEN04	Generación	Reportes	Fallo escribiendo el resumen de generación.
GEN05	Generación	Limpieza	Warning o fallo al limpiar artefactos obsoletos.
GEN06	Build	javac	Fallo ejecutando javac, build interrumpido o .class no generado correctamente.

10. Generación de código Java

La generación usa un visitor sobre el parse tree. El traductor toma el contexto del programa, visita imports, FX, funciones y project block, y acumula comentarios de cabecera, constantes, helpers y el cuerpo principal del método main.

- PROJECT -> clase Java + método main.
- DISPLAY / WRITE -> System.out.println(...).
- READ X -> lectura con helper readNumber(...).
- ID := expr -> declaración o reasignación coherente con el modelo actual.
- IF / ELSE / END -> if (...) { ... } else { ... }.
- WHILE / DO / END -> while (...) { ... }.
- FUNCTION -> método private static double ...
- Batch assignments -> múltiples declaraciones/asignaciones con metadata de unidad.

11. Built-ins y runtime financiero

El runtime Java documenta y utiliza un modelo consistente para dinero y built-ins. USD actúa como representación interna del tipo money; GTQ y EUR se convierten con FX al momento de almacenar o presentar, según corresponda.

- NPV(rate, capex, cf1..cfn) implementa el valor presente neto usando capex como inversión inicial y flujos descontados.
- PAYBACK(capex, cf1..cfn) calcula el período de recuperación acumulando flujos; puede devolver fracción de período.
- toUsd(...) convierte a la representación interna.
- fromUsd(...) convierte desde USD hacia GTQ o EUR para salida.
- formatMoney(...) aplica formato amigable al valor mostrado.
- DISPLAY expr in GTQ afecta presentación, no mutación del valor almacenado.

12. Build, execution y experiencia de consola

Después de generar el archivo Java, el compilador ejecuta el build del .java generado con javac y, para programas válidos, realiza la execution del resultado.

Etiqueta	Uso
[STATUS]	Indica parse ok/failed y semantic ok/failed.
[SYMBOLS]	Ruta del CSV exportado.
[TREE]	Ruta del DOT/SVG generado.
[JAVA]	Ruta del archivo .java generado.
[BUILD]	Resultado del build del Java generado, normalmente .class.
[EXECUTION]	Estado de la ejecución del programa generado.
[ACTION]	Mensaje positivo del compilador cuando completa la generación.

13. Estructura profesional de salida

```
output/  
  java/      # archivos .java generados  
  classes/   # archivos .class generados por build  
  trees/     # árboles .dot y .svg  
  symbols/   # tablas de símbolos .csv  
  reports/   # resúmenes y logs opcionales
```

Esta estructura reemplaza el uso de testing/ como destino por defecto cuando el compilador se ejecuta fuera del repositorio.

14. Jar final, entry point y comandos de uso

Artefacto final: app/target/bickspec-compiler-1.0.0.jar

Main-Class configurada: com.bickspec.app.TranspileRunner

Comando funcional de referencia (general):

```
java -jar app/target/bickspec-compiler-1.0.0.jar input/Archivo.bks
```

Comandos típicos adicionales:

```
java -jar app/target/bickspec-compiler-1.0.0.jar testing/P6_Imports_NPV.bks
```

```
java -jar app/target/bickspec-compiler-1.0.0.jar testing
```

Para ejecutar el compilador fuera del IDE se requiere Java instalado. Para el build automático del Java generado se necesita JDK en PATH, no solo JRE. Graphviz es opcional y solo afecta la generación de SVG.

15. Suite de pruebas vigente

Prueba	Tipo	Cobertura principal
P1	Positiva	Hola Mundo.
P2	Positiva	Variables, asignación, aritmética y display.
P3	Positiva	Import, FX, READ, IF/ELSE y conversión to USD. Interactiva.
P4	Positiva	Function, parámetros, llamada y money literals.
P5	Positiva	READ, WHILE y metadata de tiempo. Interactiva.
P6	Positiva	Imports, FX, built-in NPV y DISPLAY expr in GTQ.
P7	Positiva	Batch assignment monetario.
P8	Positiva	Batch assignment temporal.
P9	Negativa	Fallo léxico LEX01.
P10	Negativa	Fallo sintáctico SYN01.
P11	Negativa	Fallo semántico SEM01.
P12	Positiva	Recursividad y traducción de función recursiva.

16. Estado final de la Fase III

El proyecto puede considerarse funcionalmente completo en su ciclo principal. BickSpec cuenta con especificación formal, compilador ejecutable, semántica validada, generación automática de Java, build y execution del resultado.

- Cumple con la recepción de un archivo fuente y la generación de Java como salida.
- Bloquea la generación si existen errores léxicos, sintácticos o semánticos.
- Usa tabla de símbolos y validación semántica real.
- Integra built-ins financieros y modelo monetario documentado.
- Puede ejecutarse desde terminal por medio del jar final.

Las extensiones futuras ya no pertenecen al núcleo mínimo del compilador, sino a una capa opcional o de producto.